Reagan Morgan

CS-300

Project One

VECTOR PSUEDOCODE:

Program start

Struct Course {

Initialize courseNumber;

Initialize courseName;

Initialize vector prereqs;

}

Main() {

Vector<Course> courseVector;

Load CSV file

Loop CSV file:

Create new course, input first column into courseNumber, second column into courseName;

Create new vector prereqs = course vector prereqs;

```
                    If row contains more than 2 fields: insert each following field into
                    prereqs;
                    CourseVector.append(Course);


Void Search(String courseNumber) {
        Vector<Course> tempVec = courseVector;
        Loop through tempVec:
                If course.courseNumber == tempVec[i].courseNumber:
                    Print courseNumber + courseName;
                    Print vector<string> prereqs;
}
Void Print() {
        Using std library sort function, sort courseVector.
        Loop through courseVector:
                Print courseNumber + courseName;
        }
}
```

RUNTIME ANALYSIS VECTOR SEARCH:

Code	Line Cost	# Times Executes	Total Cost
Vector<Course> tempVec = courseVector;	1	1	1
Loop through tempVec:	1	n	n
If course.courseNumber == tempVec[i].courseNumber:	1	n	n
Print courseNumber + courseName;	1	1	1
Print vector<string> prereqs;	1	n	n
		Total Cost:	3n + 2
		Runtime:	O(n)

HASHTABLE PSUEDOCODE:

```
Program start

Struct Course {
        Initialize courseNumber;
        Initialize courseName;
        Initialize vector prereqs;
}

Class HashTable {
        private:
        Struct Node {
                Initialize Course course;
                Initialize Unsigned int key;
                Initialize Node *next;
                Construct Node() {
                        Assign key to UINT_MAX;
                        Assign next to NULL;
```

```
        }
        Construct Node(Course course, Unsigned int key) {
            Assign key to key;
            Assign course to course;
        }
    }
    vector nodes;
}
Main() {
Load CSV file
    Loop CSV file:
        Create new course, input first column into courseNumber, input second
        column into courseName;
        Create new vector prereqs;
        If row contains more than 2 fields: insert each following field into prereqs
        Create key for courseNumber;
        If nodes.at(key).key == UINT_MAX {
            Node newNode = new Node(course, key);
```

```
            }

            Node* cursNode = &nodes.at(key)

            Node* newNode = new Node(course, key);

            Else {

                  While (cursNode->next != NULL){

                        cursNode = cursNode->next;

                  }

                  cursNode->next = newNode;

            }

            Else:

                  Continue to next row;

}

Void Search(string courseNumber)

      Create key for courseNumber;

      Node* currNode = &nodes.at(key);

      While (currNode != NULL) {

      If (currNode's course courseNumber == courseNumber)
```

Print courseNumber;

Print courseName;

Print course vector<string> prereqs;

```
Void Print() {

Use standard library sort function to sort vector of nodes.

Loop through vector:

        Node* currNode = &nodes.at(i);

        If (currNode->key != UINT_MAX) {

            Print courseName & courseNumber;

        While (currNode->next != NULL) {

                currNode = currNode->next;

        Print courseName & courseNumber;

        }

        }

    }
```

Code	Line Cost	# Times Executes	Total Cost
Create key for courseNumber;	1	1	1
Node* currNode = &nodes.at(key);	1	1	1
While (currNode != NULL) {	1	n	n
If (currNode's course courseNumber == courseNumber)	1	n	n

Print courseNumber + courseName;	1	1	1
Print course vector<string> prereqs;	1	n	n
		Total Cost:	3n + 3
		Runtime:	O(n)

BINARY SEARCH TREE PSUEDOCODE:

```
Program Start
    Struct Course {
    Initialize courseNumber;
    Initialize courseName;
    Initialize vector<string> prereqs;
    }
    Class BinaryTree {
        Private:
        Struct Node {
            Initialize Course course;
            Initialize Node *left;
            Initialize Node *right;

            Construct Node() {
```

Assign left to nullptr;

Assign right to nullptr;

}

Construct Node(Course *course*) {

Assign course to *course;*

}

}

Node* root;

}

Main() {

Load CSV file

Loop CSV file:

Create new course, input first column into courseNumber, input second column into courseName;

If row contains more than 2 columns:

Create new vector<string> prereqs;

Loop through remaining columns and place each into

Vector<string> prereqs

Call Insert(Course course);

Void Insert(Course *course)* {

Create node Node* currNode = root;

If (currNode is nullptr) {

Assign root to new Node(*course*);

}

Else {

While (currNode is not nullptr) {

If (course's courseNumber compared to currNode's course

courseNumber) {

Void Search(String *courseNumber*) {

Create Node* currNode = root;

```
        While (currNode != nullptr) {
                If (currNode's course courseNumber is equal to courseNumber) {
                        Print courseNumber;
                        Print courseName;
                        Print course vector prereqs;
                }

                Else if (currNode's course courseNumber comapred to couseNumber is < 0)
                {
        currNode = currNode->right;
                }
        Else if (currNode's course courseNumber compared to courseNumber is > 0)
                {
                CurrNode = currNode->left;
                }
        }
}
```

```
Void Print() { //mimicing the in-order idea

Create node* currNode = root;

While (currNode != null) {

        Print(currNode->left);

        Print courseNumber + courseName;

        Print(currNode->right);

    }

}
```

RUNTIME ANALYSIS VECTOR:

Code	Line Cost	# Times Executes	Total Cost
Create node* currNode = root;	1	1	1
While (currNode != nullptr) {	1	n	n
If currNode's course courseNumber == courseNumber {	1	n	n
Print courseNumber + courseName;	1	1	1

Print course vector prereqs;	1	n	n
Else if (currNode's course courseNumber compared to courseNumber is < 0	1	n	n
CurrNode = currNode->right	1	1	1
Else if (currNode's course courseNumber compared to courseNumber is > 0	1	n	n
CurrNode = currNode->left;	1	1	1
		Total Cost:	5n + 4
		Runtime:	O(n)

Menu Psuedocode:

Main() {

Int selection;

Take user input for selection;

Switch selection:

Selection = 1:

If Vector:

Load CSV file into vector

If HashTable:

Load CSV file into HashTable using previous HashTable psuedocode in main

If BST:

Load CSV file into HashTable using previous BST psuedocode in main

Selection = 2:

If Vector:

Use print from vector Psuedocode

If HashTable:

Use print from hashTable psuedocode

If BST:

Use print from BST psuedocode

Selection = 3:

If Vector:

Use search from vector psuedocode

If hashTable:

Use search from hashtable psuedocode

If BST:

Use search from BST psuedocode

Selection = 9:

Return;

}

Recommendation:

Given the runtime analysis for each, their worst case scenarios are relatively similar. But a BST has an easier sorting process with recursion, without the need to loop through each part of the vector. Even when using something like quickSort on a hashtable, the worst case scenario can go to O(n^2). Compared to In-order traversal, which has a worst case of O(n). Because of this, I am going with implementing a BST for my final project.